RSpec can do a couple of other useful things with return values. If the method being stubbed takes a block and you want to cause the stubbed method to yield a particular set of arguments to the block, you can do so with and_yield:

```
`allow(project).to receive(:method).and_yield("arg")`
```

The expectation is that some method takes a block argument, and you want to pass through method and send arg to the block.

You can also cause the original method to be called as normal with and_call_original. You might use and_call_original if you're just spying on the method to see if it is called but you want the original return value for some reason. Alternatively, you can use it in conjunction with with to specify default behavior for most arguments and special behavior for some arguments. More rarely, there's and_wrap_original, which takes a block argument and runs the return value of the original method through the block before continuing.

Finally, you can pass a block to receive, in which case the block is executed to be the method's value:

```
first_name = "Noel"
last_name = "Rappin"
allow(project).to receive(:name) { first_name + last_name }
```

In theory this gives you some flexibility to manage the method's output. In practice it may mean it's time to abandon RSpec mocks in favor of a dummy object via OpenStruct, or that it's time to rethink the code design.

Mocks with Arguments

You can tune an RSpec double to return different values based on the parameters passed to the method using the with method to filter incoming calls. In its simplest form, shown earlier in the chapter, the with method takes one or more arguments. When the stubbed method is called, RSpec searches for a match between the arguments passed and the declared double and returns the value matching those arguments. Take care; by setting expectations tied to specific input values, you're limiting the RSpec double to only those input values. For instance, the earlier example of stubbing methods calls find(1). If you change the double to expect a different number, the test will fail:

```
it "stubs the class" do
  allow(Project).to receive(:find).with(3).and_return(
      Project.new(:name => "Project Greenlight"))
  project = Project.find(1)
  expect(project.name).to eq("Project Greenlight")
end
```